

OWASP Top 10

Insecure Design

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

Insecure Design

Índice

- Introducción
- ¿Qué significa "Insecure Design"?
- ¿Por qué ocurre esta vulnerabilidad?
- Ejemplos reales y típicos
- ¿Qué puede hacer un atacante si explota esto?
- ¿Cómo detectarlo?
- ¿Cómo prevenirlo?
- ¿Cómo mitigar si ya estás afectado?
- Checklist rápido
- Conclusiones



Insecure Design

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

Introducción

- Fallos en arquitectura, requisitos o flujos de negocio
- No es bug de código: es debilidad estructural desde el diseño
- Compromete la seguridad incluso con implementación perfecta
- Objetivo: integrar seguridad desde el primer boceto

¿Qué significa "Insecure Design"?

- Decisiones de diseño que omiten controles de seguridad
- Requisitos que priorizan UX/velocidad sobre protección
- Ausencia de modelado de amenazas, límites o trazabilidad
- Regla clave: seguridad desde el inicio, no como parche

¿Por qué ocurre esta vulnerabilidad?

- Falta de threat modeling en etapas tempranas
- Roles, permisos o scopes mal definidos
- Arquitectura que expone capacidades innecesarias
- UX sin controles compensatorios (MFA, rate limiting)
- Ausencia de patrones: least privilege, fail-safe defaults

Ejemplos reales y típicos

- Recursos compartidos sin expiración (enlaces, tokens perpetuos)
- Roles ambiguos: "usuario" con acciones administrativas
- APIs sin scopes ni límites: extracción masiva sin control
- Lógica explotable: combinar acciones para saltar validaciones
- Onboarding sin autenticación: fricción baja, riesgo alto

¿Qué puede hacer un atacante si explota esto?

- Acceder a datos sensibles mediante flujos legítimos mal protegidos
- Escalar privilegios combinando acciones permitidas
- Abusar de integraciones para extracción o movimiento lateral
- Ejecutar ataques automatizados por falta de throttling
- Evadir detección sin logs o trazabilidad diseñada

¿Cómo detectarlo?

- Requisitos: historias sin criterios de seguridad
- Arquitectura: ausencia de threat modeling documentado
- Desarrollo: "parches" de seguridad añadidos tarde
- Producción: abuso recurrente, roles con acciones no previstas
- Señales: APIs sin límites, tokens perpetuos, auditoría ausente

¿Cómo prevenirlo?

- Threat modeling por feature (actores, assets, abusos)
- Security by Design: seguridad en Definition of Done
- Least privilege y fail-safe: denegar por defecto
- Contratos API con scopes, límites y expiración
- Auditable by design: logs para acciones sensibles
- Pruebas de abuso (abuse cases) en CI/CD

¿Cómo mitigar si ya estás afectado?

- Evaluar alcance: funcionalidades y usuarios afectados
- Controles inmediatos: feature flags, rate limiting, WAF
- Parches en acceso: middleware de autorización
- Restringir integraciones: ajustar scopes, revocar permisos
- Invalidar recursos inseguros: enlaces/tokens perpetuos
- Plan de re-arquitectura para eliminar causa raíz

Checklist rápido

- ✓ ¿Micro threat modeling para esta feature?
- ✓ ¿Roles y permisos con granularidad mínima?
- ✓ ¿Límites de uso (rate limiting/throttling) aplicados?
- ✓ ¿Recursos compartidos con expiración y revocación?
- ✓ ¿Integraciones con scopes finos y auditoría?
- ✓ ¿Logs para acciones críticas?
- ✓ ¿Plan de rotación para claves/dependencias?
- ✓ ¿Pruebas de abuso o lógica de negocio incluidas?

Conclusiones

1. Insecure Design es estructural: se previene, no se parchea
2. Seguridad integrada al ciclo de producto desde el inicio
3. Threat modeling, least privilege y auditoría son esenciales
4. Como desarrollador: cuestionar, modelar rápido y validar lógica
5. Diseñar bien hoy evita incidentes costosos mañana



¡MUCHAS GRACIAS!

BIG school

BIGSEIO

**GENERAMOS
NEGOCIO**